

Bakalářské zkoušky (příklady otázek)

2026-02-02

1 Porovnání modulárních hashů (společné okruhy)

Mějme dané prvočíslo $p \geq 2$. Pro slovo $w \in \{0, 1\}^*$ nechť $h(w)$ je zbytek modulo p z nezáporného celého čísla, jehož binární reprezentace je w , přičemž povolujeme úvodní nuly. Přesněji, definujeme funkci $h : \{0, 1\}^* \rightarrow \{0, 1, \dots, p-1\}$ následovně:

$$h(w) = \left(\sum_{i=1}^{|w|} w_i 2^{|w|-i} \right) \bmod p,$$

kde $w = w_1 w_2 \dots w_{|w|}$ a $w_i \in \{0, 1\}$. (Pro prázdné slovo je tedy $h(\varepsilon) = 0$.)

Nyní uvažte následující jazyk nad abecedou $\Sigma = \{0, 1, \#\}$:

$$L_p = \{ u\#v \mid u, v \in \{0, 1\}^* \text{ a platí } h(u) = h(v) \}$$

1. Uveďte formální definici *deterministického konečného automatu (DFA)*.
2. Formálně popište konstrukci DFA A_p , který rozpoznává jazyk L_p , v závislosti na parametru p .
3. Nakreslete stavový diagram automatu A_2 (rozpoznávajícího jazyk L_2).

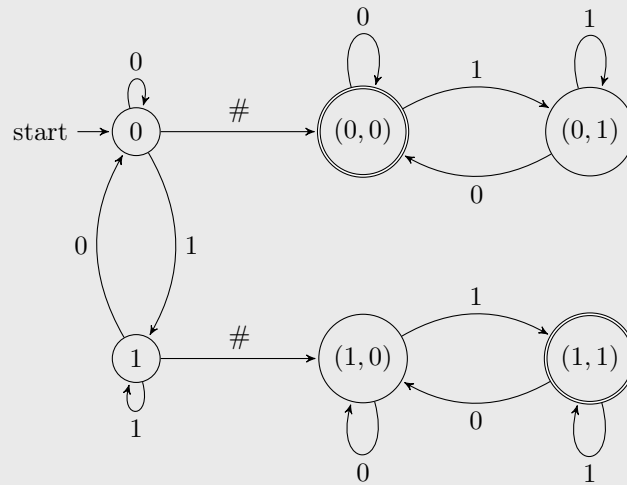
Nástin řešení

1. DFA je pětice $A = (Q, \Sigma, \delta, q_0, F)$, kde:
 - Q je konečná neprázdná množina stavů,
 - Σ je konečná neprázdná abeceda,
 - $\delta : Q \times \Sigma \rightarrow Q$ je (parciální) přechodová funkce,
 - $q_0 \in Q$ je počáteční stav,
 - $F \subseteq Q$ je množina přijímajících stavů.
2. DFA $A_p = (Q, \Sigma, \delta, q_0, F)$ zkonstruujeme takto:
 - $Q = [p] \cup [p] \times [p]$ kde $[p] = \{0, 1, \dots, p-1\}$ (stav i znamená, že jsme přečetli prefix u' slova u a $i = h(u')$, stav (i, j) znamená, že jsme přečetli prefix v' slova v , $i = h(u)$ a $j = h(v')$).
 - $\Sigma = \{0, 1, \#\}$,
 - $q_0 = 0$,
 - $F = \{(i, i) \mid i \in \{0, 1, \dots, p-1\}\}$,
 - přechodová funkce δ je definována takto (kde $i, j \in [p]$):
 - $\delta(i, 0) = 2i \bmod p$,
 - $\delta(i, 1) = (2i + 1) \bmod p$,
 - $\delta(i, \#) = (i, 0)$,
 - $\delta((i, j), 0) = (i, 2j \bmod p)$,

$$- \delta((i, j), 1) = (i, (2j + 1) \bmod p),$$

a zbývající přechody jsou nedefinované (anebo vedou do implicitního mrtvého stavu, tzv. žumpy).

3. Stavový diagram automatu A_2 :



2 Semafor mezi procesy (společné okruhy)

Implementujte dvojici funkcí s tímto rozhraním:

```
void send(byte ch);
byte receive();
```

Účelem těchto funkcí je realizace jednosměrného proudu bajtů mezi dvěma procesy. Součástí proudu je FIFO-buffer s kapacitou N bajtů. Funkce `receive`, volaná procesem příjemce, čeká, dokud v bufferu není alespoň jeden platný bajt, poté tento bajt odebere a vrátí jej. Funkce `send`, volaná z procesu odesilatele, čeká, dokud v bufferu není alespoň jedna volná pozice, poté do bufferu vloží parametr `ch`.

Vášim úkolem je implementovat funkce `send` a `receive` pouze pomocí těchto součástí:

- Semaforey (pracující přes hranice procesů)
- Implementace bufferu s kapacitou N , která poskytuje následující rozhraní:

```
void buffer_push(byte ch);
byte buffer_pop();
```

Funkce `buffer_push` (volaná procesem odesilatele) a funkce `buffer_pop` (volaná procesem příjemce) jsou atomické (tj. mohou být bezpečně volány současně) a umožňují předávání bajtů přes hranice procesů (např. pomocí sdílené paměti a atomických instrukcí). Tyto funkce však neposkytují synchronizaci (vždy se vracejí okamžitě). Funkce `buffer_push` nesmí být volána, pokud je buffer plný, a funkce `buffer_pop` nesmí být volána, pokud je buffer prázdný.

Vášim úkolem není řešit vytvoření semaforů a bufferu, předpokládejte, že oba procesy mohou přistupovat k těmto objektům, jako by existovaly uvnitř těchto procesů.

1. Napište implementaci funkcí `send` a `receive` v některém z jazyků C, C++, C# nebo Java. Implementace nesmí používat žádné statické nebo globální proměnné ani žádné další knihovny kromě semaforů a výše popsaného bufferu. Implementace nesmí využívat aktivní čekání (polling). Specifikujte také počáteční stav(y) semaforu/ů (v závislosti na kapacitě bufferu N); buffer je na počátku prázdný.
2. Definujte invariant(y) platné pro stav(y) semaforu/ů ve vztahu k počtu platných elementů v bufferu (a jeho kapacitě N). Všechny tyto invarianty musí být platné po celou dobu mezi voláními funkcí `send` nebo `receive`.
3. Vysvětlete, jak se platnost invariantů mění uvnitř funkcí `send` a `receive` (např. pomocí pozměněných invariantů platných mezi jednotlivými příkazy uvnitř těchto funkcí). Pro jednoduchost předpokládejte, že tyto funkce nejsou volány zároveň.

Nástin řešení

```
// capacity_semaphore initialized to N
// content_semaphore initialized to 0
//
// Invariant 1: capacity_semaphore = N - elements_in_buffer
// Invariant 2: content_semaphore = elements_in_buffer

void send(char ch)
{
    capacity_semaphore.down();
    // Inv 1 skewed by +1, Inv 2 valid
    buffer_push(ch);
    // Inv 1 valid, Inv 2 skewed by -1
    content_semaphore.up();
}

char receive()
{
    content_semaphore.down();
    // Inv 1 valid, Inv 2 skewed by +1
    char ch = buffer_pop();
    // Inv 1 skewed by -1, Inv 2 valid
    capacity_semaphore.up();
}
```

3 Matice (společné okruhy)

Uvažujme matici $A \in \mathbb{R}^{3 \times 3}$, jejíž řádkový prostor má bázi tvořenou pouze vektorem $(1, 2, 3)^T$.

1. Najděte odstupňovaný tvar matice A .
2. Napište definici determinantu a spočítejte konkrétní hodnotu $\det(A)$.
3. Najděte ortogonální bázi jádra matice A .

Nástin řešení

1.

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

2.

$$\det(A) = \sum_{p \in S_n} \operatorname{sgn}(p) \prod_{i=1}^n a_{i,p(i)}.$$

V našem případě má matice A hodnotu 1, čili je singulární a $\det(A) = 0$.

3. Jádro matice A má dimenzi 2 a bázi tvoří např. vektory $(2, -1, 0)^T$, $(3, 0, -1)^T$. Gramovou-Schmidtovou ortogonalizací dostaneme ortogonální bázi $(2, -1, 0)^T$, $(3, 6, -5)^T$.

4 Rovinné grafy (společné okruhy)

1. Uveďte Eulerovu formuli pro rovinné grafy (o počtu vrcholů, hran a stěn).
2. Uvažujme vrcholově 2-souvislý rovinný graf, který má rovinné nakreslení, jehož každá vnitřní stěna je trojúhelník (je ohraničena kružnicí C_3), vnější stěna může, ale nemusí být ohraničená C_3 . Jaký je minimální a maximální počet hran

takového grafu, pokud víme, že má $n \geq 3$ vrcholů? Odpověď přiměřeně zdůvodněte.

3. Rozhodněte, pro která $n \in \mathbb{N}$ existuje rovinný graf s $2n$ vrcholy takový, že n z nich má stupeň 3 a zbývajících n má stupeň 4. Odpověď přiměřeně zdůvodněte.

Nástin řešení

1. Je-li $G = (V, E)$ souvislý rovinný graf a s počet stěn nějakého jeho rovinného nakreslení, pak $|V| - |E| + s = 2$.
2. Z 2-souvislosti plyne, že každá stěna je ohraničena kružnicí a každá hrana leží na hranici 2 různých stěn. Je-li vnější stěna ohraničena kružnicí C_k ($3 \leq k \leq n$), máme $2|E| = 3(s - 1) + k$. Použitím Eulerovy formule dostáváme $|E| = 3n - 3 - k$.

Maxima se nabývá v případě, že i vnější stěna je ohraničena C_3 a to $3n - 6$ hran. Minima pak pro vnějškově rovinný graf, který je vnitřně triangulovaný. Ten má $2n - 3$ hran.

3. Protože graf obsahuje vrchol stupně 4, musí mít alespoň 5 vrcholů a $n \geq 3$. Z principu sudosti plyne, že má $\frac{3n+4n}{2}$ hran a tedy n musí být sudé.

Dokázat, že pro všechna sudá $n \geq 4$ takový graf existuje lze konstrukcí. Rovinnost nepřidává žádné další omezení, jen je třeba ji zohlednit při konstrukci. Pozor, konstrukce musí být dostatečně obecná, ne jen pro několik menších případů n .